

Objective

The objective of this lab experiment is to:

1. Implement Minimum Spanning Tree algorithms: Prim's and Kruskal's.
2. Implement Shortest Path algorithms: Dijkstra's and Bellman-Ford.
3. Compare the performance, logic, and limitations of each algorithm.
4. Understand how different algorithms work for graph problems.

Background Study:

1. Prim's Algorithm:

Builds the MST by starting from a vertex and adding the smallest edge connecting a visited vertex to an unvisited one.

Uses a greedy approach.

2. Kruskal's Algorithm:

Sorts all edges by weight and selects the smallest edge that doesn't form a cycle.

Uses Disjoint Set Union (DSU) for cycle detection.

3. Dijkstra's Algorithm:

Finds the shortest path from a source to all other vertices.

Works only for non-negative edge weights.

Uses a priority queue for efficiency.

4. Bellman-Ford Algorithm:

Works with negative weights and detects negative weight cycles.

Runs in $O(V \cdot E)$ time by relaxing all edges $V-1$ times.

Algorithm Complexities:

Algorithm	Time Complexity	Works with Negative Weights	Detects Negative Cycles.
Prim's (adj list, min heap)	$O(E \log V)$	No	No
Kruskal's	$O(E \log E)$	No	No
Dijkstra's	$O(E \log V)$	No	No

Bellman-Ford $O(V \cdot E)$ Yes Yes

Code:

```
#include <iostream>

#include <vector>

#include <queue>

#include <algorithm>

#include <climits>

Using namespace std;

Struct Edge {

Int u, v, w;

Edge(int a, int b, int c) : u(a), v(b), w(c) {}

};

Class Graph {

Public:

Int V;

Vector<Edge> edges;           // For Kruskal and Bellman-Ford

Vector<vector<pair<int,int>>> adj; // adjacency list for Prim & Dijkstra

Graph(int vertices) {

    V = vertices;

    Adj.resize(V);

}

Void addEdge(int u, int v, int w) {

    Edges.push_back(Edge(u, v, w));

    Adj[u].push_back({v, w});

    Adj[v].push_back({u, w}); // undirected

}

// --- Prim's MST ---

Void prims() {
```

```

Cout << "Prim's MST:\n";

Vector<int> key(V, INT_MAX);

Vector<bool> inMST(V, false);

Vector<int> parent(V, -1);

Key[0] = 0;

For (int count = 0; count < V - 1; count++){

    Int u = -1;

    Int minKey = INT_MAX;

    For (int i = 0; i < V; i++) {

        If (!inMST[i] && key[i] < minKey) {

            minKey = key[i];

            u = i;

        }

    }

    inMST[u] = true;

    for (auto &p : adj[u]) {

        int v = p.first, w = p.second;

        if (!inMST[v] && w < key[v]) {

            key[v] = w;

            parent[v] = u;

        }

    }

}

For (int i = 1; i < V; i++) {

    Cout << parent[i] << " - " << i << " : " << key[i] << "\n";

}

Cout << "\n";

}

// --- Kruskal's MST ---

```

```

Int findSet(vector<int> &parent, int i) {
    If (parent[i] != i)
        Parent[i] = findSet(parent, parent[i]);
    Return parent[i];
}

Void unionSet(vector<int> &parent, vector<int> &rank, int x, int y) {
    Int xroot = findSet(parent, x);
    Int yroot = findSet(parent, y);
    If (xroot != yroot) {
        If (rank[xroot] < rank[yroot])
            Parent[xroot] = yroot;
        Else if (rank[yroot] < rank[xroot])
            Parent[yroot] = xroot;
        Else {
            Parent[yroot] = xroot;
            Rank[xroot]++;
        }
    }
}

Void kruskal() {
    Cout << "Kruskal's MST:\n";
    Vector<Edge> result;
    Sort(edges.begin(), edges.end(), [](Edge a, Edge b){ return a.w < b.w; });
    Vector<int> parent(V);
    Vector<int> rank(V, 0);
    For (int i = 0; i < V; i++) parent[i] = i;
    Int e = 0, i = 0;
    While (e < V - 1 && i < (int)edges.size()) {
        Edge next_edge = edges[i++];
    }
}

```

```

Int x = findSet(parent, next_edge.u);

    Int y = findSet(parent, next_edge.v);
If (x != y) {
    Result.push_back(next_edge);
    unionSet(parent, rank, x, y);
    e++;
}
}

For (auto &edge : result) {
    Cout << edge.u << " - " << edge.v << " : " << edge.w << "\n";
}

Cout << "\n";
}

// --- Dijkstra's shortest paths ---

Void dijkstra(int src) {
    Cout << "Dijkstra's shortest paths from node " << src << ":\n";
    Vector<int> dist(V, INT_MAX);
    Dist[src] = 0;
    // min heap (distance, vertex)
    Priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> pq;
    Pq.push({0, src});
    While (!pq.empty()) {
        Int u = pq.top().second;
        Int currDist = pq.top().first;
        Pq.pop();
        If (currDist > dist[u]) continue;
        For (auto &p : adj[u]) {
            Int v = p.first, w = p.second;
            If (dist[u] != INT_MAX && dist[u] + w < dist[v]) {

```

```

        Dist[v] = dist[u] + w;

        PQ.push({dist[v], v});
    }
}

}

For (int i = 0; i < V; i++) {

    Cout << "Distance from " << src << " to " << i << ": ";

    If (dist[i] == INT_MAX) cout << "INF";

    Else cout << dist[i];

    Cout << "\n";

}

Cout << "\n";

}

// --- Bellman-Ford shortest paths ---

Void bellmanFord(int src) {

    Cout << "Bellman-Ford shortest paths from node " << src << ":\n";

    Vector<int> dist(V, INT_MAX);

    Dist[src] = 0;

    For (int i = 1; i <= V - 1; i++) {

        For (auto &e : edges) {

            Int u = e.u, v = e.v, w = e.w;

            If (dist[u] != INT_MAX && dist[u] + w < dist[v]) {

                Dist[v] = dist[u] + w;

            }

        }

    }

}

// Check for negative cycle

For (auto &e : edges) {

    Int u = e.u, v = e.v, w = e.w;

```

```

    If (dist[u] != INT_MAX && dist[u] + w < dist[v]) {

        Cout << "Graph contains negative weight cycle\n\n";

        Return;

    }

}

For (int i = 0; i < V; i++) {

    Cout << "Distance from " << src << " to " << i << ": ";

    If (dist[i] == INT_MAX) cout << "INF";

    Else cout << dist[i];

    Cout << "\n";

}

Cout << "\n";

}

};

Int main() {

Graph g(5);

g.addEdge(0, 1, 2);

g.addEdge(0, 3, 6);

g.addEdge(1, 2, 3);

g.addEdge(1, 3, 8);

g.addEdge(1, 4, 5);

g.addEdge(2, 4, 7);

g.addEdge(3, 4, 9);

// Call Prims

g.prim();

// Call Kruskal

g.kruskal();

// Call Dijkstra

g.dijkstra(0);

```

```

// Update one edge to negative weight before Bellman-Ford

// Change edge 3-4 weight to -10

For (auto &e : g.edges) {

    If ((e.u == 3 && e.v == 4) || (e.u == 4 && e.v == 3)) {

        e.w = -10;

    }

}

// Call Bellman-Ford

g.bellmanFord(0);

return 0;

}

```

Output:

Prim's MST:

0 – 1 : 2

1 – 2 : 3

0 – 3 : 6

1 – 4 : 5

Kruskal's MST:

0 – 1 : 2

1 – 2 : 3

1 – 4 : 5

0 – 3 : 6

Dijkstra's shortest paths from node 0:

Distance from 0 to 0: 0

Distance from 0 to 1: 2

Distance from 0 to 2: 5

Distance from 0 to 3: 6

Distance from 0 to 4: 7

Bellman-Ford shortest paths from node 0:

Distance from 0 to 0: 0

Distance from 0 to 1: 2

Distance from 0 to 2: 5

Distance from 0 to 3: 6

Distance from 0 to 4: -4

Conclusion:

From this experiment:

1. Prim's and Kruskal's efficiently find MSTs, but they are suited for different graph densities: Prim's for dense graphs, Kruskal's for sparse ones.
2. Dijkstra's is efficient for shortest paths when all weights are positive.
3. Bellman-Ford is slower but can handle negative weights and detect negative weight cycles.
4. In the given program, changing an edge to a negative weight made Bellman-Ford more appropriate, while Dijkstra's gave incorrect results for that case.

This lab enhanced understanding of greedy and dynamic programming approaches in graph theory.